

Software Requirements Specification

Practice Order Management System

Version 2.0 | October 8, 2026

Author: David

Status: Active Development

Revision History

Version	Date	Summary
1.0	June 27, 2026	Initial specification.
2.0	October 8, 2026	Phase 14. Prices move from long cents to long units of \$0.0001. Every order carries a time in force (GTC, IOC, FOK). Iceberg and midpoint peg order types are added, with the ORDER_EXPIRED execution type. Sections revised: 1.3, 2.1, 3.1, 3.2, 3.3, 3.4, 3.6, 4, 6. All other sections are unchanged from 1.0.

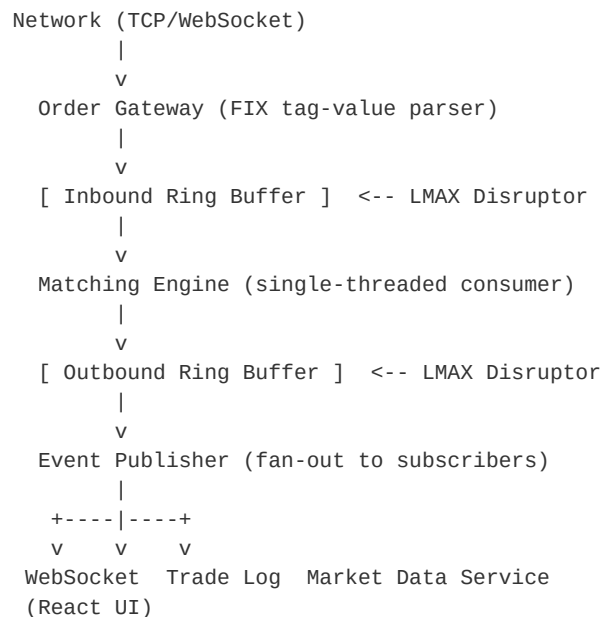
1. Project Overview

1.1 Purpose

A practice Order Management System (OMS) built in Java demonstrating low-latency systems design, event-driven architecture, concurrency patterns, and financial domain knowledge. The system implements a limit order book matching engine connected via a Disruptor-based pipeline to a FIX protocol gateway and WebSocket-powered React frontend.

This project exists as a portfolio artifact. Every architectural decision prioritizes demonstrating competence in the skills that matter for trading systems engineering: mechanical sympathy, lock-free data structures, protocol-level networking, and disciplined scope management.

1.2 High-Level Architecture



1.3 Scope Boundaries

In scope:

- Limit order matching with price-time priority
- Time in force on every order: GTC, IOC, FOK
- Iceberg orders (displayed tip, hidden reserve)
- Midpoint peg orders (non-displayed, executing at the midpoint of the venue's own lit book)
- Order submission (NewOrderSingle) and cancellation (OrderCancelRequest)
- FIX tag-value protocol parsing (subset)
- Disruptor-based event pipeline
- WebSocket push to React frontend (depth chart, trade tape)
- Latency benchmarking with JMH

Explicitly out of scope:

- Spring Boot or any DI framework
- AI trading agents
- Full FIX session management (heartbeats, sequence recovery, logon/logout)
- Market order types, stop orders
- Day and GTD time in force (the venue has no session clock for them to expire against)
- Fully hidden (zero-display) limit orders
- Peg price caps, peg offsets, and primary or market peg types
- Per-participant routing of execution reports (see §3.6)
- Multi-asset or multi-symbol support
- Persistence / database layer
- Authentication / user management

2. Finalized Tech Stack

Technology	Role	Justification
Java	Language	Full control over memory layout, threading, allocation. No runtime magic or reflection overhead.
Maven	Build	Already in use, reliable, no reason to change.
LMAX Disruptor	Pipeline	Lock-free, pre-allocated ring buffers replacing blocking queues. Eliminates GC pressure on the hot path.
Netty	Network / WebSocket	Non-blocking, event-loop driven. Industry standard for trading system network I/O.
Hand-rolled FIX parser	Protocol	Parses FIX subset (35=D, 35=F). Demonstrates protocol understanding without QuickFIX/J overhead.
Jackson	Serialization	JSON at the WebSocket boundary only. Stays off the hot path.
JUnit 5	Testing	Unit and integration testing.
JMH	Benchmarking	Latency/throughput benchmarks with proper JIT warmup. Produces p50, p99 metrics.
SLF4J	Logging	Logging at boundaries only. Never on the matching engine's critical path.
React	Frontend	Live depth chart, trade tape, manual trading interface via WebSocket.

2.1 Technologies Explicitly Rejected

Technology	Reason for Rejection
Spring Boot	Heavyweight container, reflection-heavy DI, classpath scanning. Works against low-latency goals.
QuickFIX/J	Full FIX session engine. Overkill for parsing two message types.
BigDecimal	Heap allocation per arithmetic operation. All prices use long units of \$0.0001 (\$4).
LocalDateTime	Object allocation for timestamps. Replaced with long epoch nanoseconds.
Optional	Allocation overhead. Sentinel value (-1L) used for empty book sides.
BlockingQueue	Lock contention and object allocation. Replaced by Disruptor ring buffer.

3. System Components

3.1 Order Gateway

Responsibility: Protocol boundary. Receives raw bytes, parses FIX tag-value messages, validates fields, constructs Order objects, and publishes events onto the inbound ring buffer.

Threading model: Own thread (or Netty event loop). Sole producer on the inbound ring buffer.

FIX messages supported:

NewOrderSingle (35=D): Tags 11 (ClOrdID), 54 (Side), 38 (OrderQty), 55 (Symbol), plus the order-type and time-in-force tags below.

Tag	Name	Rule
40	OrdType	Optional. 2 = Limit, P = Pegged. Absent means Limit. Any other value is rejected.
44	Price	Required for Limit, as decimal dollars with at most two decimal places (the one-cent tick). Forbidden for Pegged.
18	ExecInst	Required for Pegged, and must be M (mid-price peg).
59	TimeInForce	Optional. 1 = GTC, 3 = IOC, 4 = FOK. Absent means GTC. Any other value, including 0 (Day) and 6 (GTD), is rejected.
111	MaxFloor	Optional. The display quantity of an iceberg order. A positive integer no greater than OrderQty (38).

Strict FIX defaults a missing tag 59 to Day. This venue has no session clock for a Day order to expire against, so a missing 59 means GTC. This is a deliberate, documented deviation. The system's own client always sends tag 59.

OrderCancelRequest (35=F): Tags 11 (ClOrdID), 41 (OrigClOrdID)

Validation: FIX message structure (SOH delimiters, tag=value format), required tag presence, field type validation, and the per-OrdType tag rules in the table above. Rejects malformed messages before ring buffer; such rejects are logged at the gateway and dropped. Domain validation delegated to Order constructor: limit prices on the one-cent tick, and the order-type and time-in-force combination rules of §3.3. A domain reject is reported to the client as ORDER_REJECTED.

3.2 Inbound Ring Buffer

Lock-free transport between Order Gateway and Matching Engine. LMAX Disruptor ring buffer with pre-allocated OrderEvent carrier objects. Single producer (gateway), single consumer (engine). OrderEvent is mutable; fields are copied in by the gateway and read by the engine. Slots are reused.

OrderEvent schema: eventType (NEW_ORDER / CANCEL_ORDER), orderId (long), side (Side enum), ordType (LIMIT / PEG_MID), tif (GTC / IOC / FOK), price (long units of \$0.0001; -1 for pegged orders and cancels), quantity (long), maxFloor (long; 0 when not an iceberg), timestamp (long epoch nanos),

originalOrderId (long, for cancels).

Every field a message type does not use is reset when the slot is written, so no value from a previous use of the slot can be read.

3.3 Matching Engine

Responsibility: Core order book logic. Maintains the limit order book, executes price-time priority matching, publishes fill/acknowledgment events to the outbound ring buffer.

Threading model: Single-threaded. Sole consumer on the inbound ring buffer. No locks, no synchronization, no contention. The Disruptor model provides this naturally.

Data structures: TreeMap<Long, Deque<Order>> per side (natural ordering for asks, reverseOrder for bids). HashMap<Long, Order> for O(1) cancel lookup. ArrayDeque at each price level for FIFO time priority. Two further ArrayDeque<Order> peg pools, one per side, hold resting midpoint peg orders in FIFO order, outside the price maps.

Matching rules: Price-time priority. Passive price convention (fills at resting order's price). Incoming order walks opposite side until exhausted or no matchable levels.

Time in force. Once matching is complete, any unfilled remainder is handled by the order's time in force:

- GTC: the remainder rests on the book and is acknowledged with ORDER_ACCEPTED.
- IOC: the remainder never rests. It expires and is reported with ORDER_EXPIRED.
- FOK: before any change to the book, the engine computes the quantity available to the order: resting quantity at prices within its limit, counting the full size of iceberg orders including their reserve, plus the opposite peg pool when the order crosses the midpoint. If that covers the order, normal matching fills it completely. If not, the whole order expires with ORDER_EXPIRED and the book is left unchanged. The availability check and the matching loop use one shared eligibility rule, so they cannot disagree.

Iceberg orders. An iceberg carries a display quantity (peak) and a visible slice. A resting iceberg is matched against its visible slice only. When the slice is consumed and reserve remains, the order reloads: the slice is refreshed to the smaller of the peak and the remaining quantity, and the order moves to the back of its price level's queue, losing time priority. As an aggressor, an iceberg matches with its full remaining quantity. Depth snapshots report only visible quantity, so the reserve never appears in market data.

Midpoint peg orders. The midpoint is $(\text{best bid} + \text{best ask}) / 2$, computed from the lit book only. Pegged orders never contribute to the best bid or ask, and never appear in depth snapshots. The rules are:

1. An incoming lit order that crosses the midpoint (a buy priced at or above it, a sell priced at or below it) first fills against the opposite peg pool at the midpoint, in FIFO order, because the midpoint is a better price than any lit level on that side. It then continues into the lit book as normal. The midpoint is fixed for the duration of that order's peg fills.
2. An incoming pegged order fills against the opposite peg pool at the midpoint, in FIFO order. It never takes lit liquidity, since no lit order rests at the midpoint.
3. No-reference guard: when either lit side is empty there is no midpoint. A GTC pegged order rests inactive in its pool, and an IOC or FOK pegged order expires immediately.

4. When an order rests on a previously empty lit side and so creates a midpoint, and both peg pools hold orders, the pools cross at the new midpoint, head against head, until one pool is empty. In these fills the later-arriving pegged order is the aggressor.
5. Resting lit orders and resting pegged orders can never cross each other, because every resting lit bid is below the midpoint and every resting lit ask is above it. No other re-check is needed when the midpoint moves.
6. Repricing a pegged order costs nothing: every resting pegged order is priced at the same midpoint, so their relative order never changes and no order is reinserted.

Order type and time in force combinations:

Order type	GTC	IOC	FOK
Limit	Remainder rests	Remainder expires	Fills completely or expires
Iceberg	Rests with tip and reload	Rejected	Rejected
Midpoint peg	Rests (inactive while no midpoint)	Midpoint fills, then expires	Fills completely from the peg pool or expires

An iceberg whose display quantity equals its order quantity hides nothing and is treated as a plain limit order. An iceberg that is also pegged is rejected, because a display quantity has no meaning on a non-displayed order.

3.4 Outbound Ring Buffer

Lock-free transport from Matching Engine to downstream consumers. Single producer (engine), multiple independent consumers each with its own sequence counter. Adding a subscriber requires only registering a new consumer — no engine changes.

Event types: ORDER_ACCEPTED, ORDER_FILLED, ORDER_PARTIALLY_FILLED, ORDER_CANCELLED, ORDER_REJECTED, ORDER_EXPIRED.

ExecutionEvent schema: eventType, orderId, tradeId, price (long units of \$0.0001: the fill price for fills, the limit price for ORDER_ACCEPTED, -1 for an accepted pegged order), filledQuantity, remainingQuantity, aggressorOrderId, passiveOrderId, timestamp (epoch nanos).

ORDER_EXPIRED carries orderId, and the quantity that expired unexecuted in remainingQuantity. All other fields carry -1. An IOC or FOK order produces zero or more fill events followed by exactly one ORDER_EXPIRED, and is never acknowledged with ORDER_ACCEPTED.

3.5 Event Publisher

Consumes events from the outbound ring buffer and fans them out to subscribers: WebSocket publisher (serializes to JSON via Jackson, pushes to Netty channels), market data service (best bid/ask, midpoint, spread), and trade logger (console/file trade tape).

Each subscriber is an independent consumer with its own sequence counter. Subscribers do not block each other or the matching engine.

3.6 WebSocket Layer (Netty)

Maintains persistent connections with React frontend clients. Pushes real-time events and accepts manual order submissions.

Inbound: React client sends JSON order via WebSocket. Netty handler deserializes with Jackson and publishes to inbound ring buffer via gateway. The new-order frame is:

```
{"type":"NEW","clOrdId":1,"side":"BUY","ordType":"LIMIT","tif":"GTC",  
  "price":1502500,"qty":100,"maxFloor":0,"symbol":"ASML"}
```

ordType is LIMIT or PEG_MID; tif is GTC, IOC or FOK; price is in units of \$0.0001, and is -1 for PEG_MID; maxFloor is 0 for an order that is not an iceberg. A missing ordType, tif or maxFloor defaults to LIMIT, GTC or 0. An unknown value for any of them is logged and the frame is dropped. The handler translates the frame into the FIX message of §3.1.

Outbound: Event publisher serializes execution events and book snapshots to JSON. Netty writes frames to all connected WebSocket channels. Protocol: ws:// (no TLS for demo). All prices on the wire are in units of \$0.0001.

Known limitation: execution events are written to every connected channel, not only to the session that owns the order. Book snapshots never show an iceberg's reserve or any pegged order, but ORDER_ACCEPTED and fill events carry an order's total remaining quantity, so any connected client can read the hidden size of an iceberg or a pegged order from the execution stream. A real venue sends execution reports only to the owning session. Closing this requires per-participant sessions and routing, which are out of scope (§1.3).

3.7 React Frontend

Components: Depth chart (live bid/ask visualization), trade tape (scrolling fill history), manual trading interface (order submission and cancellation). All data arrives via WebSocket push — no polling. Frontend maintains local state from the event stream.

4. Data Representation

Prices: All prices stored as long values in units of \$0.0001, at 10,000 units per dollar. \$150.25 = 1,502,500L. No floating point, no BigDecimal. Conversion to display format at UI boundaries only.

Limit prices must lie on the one-cent tick, a multiple of 100 units. This is enforced twice: by the FIX parser, which accepts at most two decimal places in tag 44, and by the Order constructor.

Because every lit price is a multiple of 100 units, the sum of the best bid and best ask is too, so the midpoint is always an exact multiple of 50 units. A \$100.00 bid and a \$100.01 ask give a midpoint of 1,000,050 (\$100.005). No price is ever rounded.

Tag 44 on the FIX wire remains decimal dollars. Prices on the JSON wire are in units. At the UI, a price on the cent tick is shown with two decimal places; a sub-penny price, which only a midpoint execution can produce, is shown with up to four decimal places with trailing zeros removed, for example 100.005.

Timestamps: All timestamps stored as long epoch nanoseconds. No LocalDateTime.

Identifiers: Generated by IdGenerator using separate AtomicLong counters for order, trade, and participant IDs.

Sentinel values: -1L for empty book sides (getBestBid/getBestAsk), for price fields that do not apply, and for the limit price of a pegged order. Avoids Optional allocation.

5. Design Principles

5.1 Zero Allocation on the Hot Path

The critical path (inbound ring buffer to matching engine to outbound ring buffer) must not allocate heap objects during steady-state operation. Ring buffer slots are pre-allocated. Event carriers are mutable and reused. Prices and timestamps are primitives. This eliminates GC pauses during matching.

5.2 Single-Writer Principle

Each ring buffer has exactly one producer. The inbound buffer's producer is the Order Gateway. The outbound buffer's producer is the Matching Engine. This eliminates write contention without locks.

5.3 Mechanical Sympathy

Ring buffer slots are adjacent in memory for cache-line-friendly sequential access. Disruptor pads sequence counters to prevent false sharing. The matching engine reads events sequentially, maximizing L1/L2 cache hits.

5.4 Logging Off the Hot Path

SLF4J logging occurs only at system boundaries: the Order Gateway (on message receipt) and the Event Publisher (on event dispatch). The matching engine's inner loop contains zero logging calls.

5.5 Protocol Boundary Separation

Serialization (JSON via Jackson) and protocol parsing (FIX tag-value) occur only at system edges. The core pipeline operates on Java primitives and pre-allocated objects. The matching engine has no knowledge of FIX, JSON, or WebSocket.

5.6 Scope Discipline

Features are included only if they serve the demo's core goals: low-latency architecture, event-driven design, financial domain knowledge, or full-stack integration. Over-engineering is actively resisted.

6. Testing Strategy

6.1 Unit Tests (JUnit 5)

- Matching engine: order placement, price-time priority, partial/full fills, cancel, empty book
- Matching engine, time in force: IOC remainder expiry, FOK complete fill, and FOK expiry leaving the book unchanged
- Matching engine, iceberg: visible-only depth, reload to the back of the queue, reserve counted by FOK
- Matching engine, midpoint peg: exact sub-penny midpoint price, peg priority over lit levels, no-reference guard, peg pool cross, non-display
- FIX parser: valid messages, missing tags, malformed messages, invalid fields, SOH handling
- FIX parser, order types: tags 40, 44, 18, 59 and 111, including defaults and rejects
- Event carriers: correct field copying on ring buffer slots, field reset between reuses

6.2 Integration Tests (JUnit 5)

- Full pipeline: FIX message at gateway through to execution event at subscriber
- Full pipeline, order types: IOC, FOK and a midpoint peg from FIX bytes to execution event, including a sub-penny execution price
- WebSocket round-trip: JSON order from client through to fill event pushed back
- Book state consistency: order/cancel sequence verified against expected book snapshot

6.3 Latency Benchmarks (JMH)

- Matching engine throughput: orders per second in isolation
- End-to-end latency: gateway receipt to execution event publication (p50, p99, p99.9)
- Book depth impact: matching latency as function of resting order count
- Phase 14 re-measure: Order allocation and matching throughput against the Phase 6 baselines, and the FOK availability check as a function of levels walked

7. Project Structure (Planned)

```
src/main/java/  
  Main.java  
  model/  
    Order.java, Side.java, Status.java, Trade.java  
  engine/  
    MatchingEngine.java, BookView.java  
  gateway/  
    FixParser.java, OrderGateway.java  
  event/  
    OrderEvent.java, OrderEventFactory.java  
    ExecutionEvent.java, ExecutionEventFactory.java  
  publisher/  
    EventPublisher.java, WebSocketPublisher.java, TradeLogger.java  
  market/  
    MarketDataService.java  
  net/  
    WebSocketServer.java  
  util/  
    IdGenerator.java
```

8. Phase Plan

Phase 1 — Core Matching Engine (Complete)

Fully implemented: Order, Side, Status, Trade, IdGenerator, MatchingEngine, Main, BookView, MarketDataService. Console-based order book display with spread/midpoint.

Phase 2 — Disruptor Pipeline (In Progress)

Define OrderEvent and ExecutionEvent carriers. Wire inbound and outbound ring buffers. Adapt MatchingEngine as Disruptor EventHandler. Implement OrderGateway as producer and EventPublisher as outbound consumer. End-to-end pipeline test.

Phase 3 — FIX Gateway

Implement FixParser for tag-value parsing. Support 35=D (NewOrderSingle) and 35=F (OrderCancelRequest). Wire into OrderGateway. Comprehensive unit tests.

Phase 4 — Netty WebSocket Server

Stand up Netty WebSocket server. Implement WebSocketPublisher subscribing to outbound ring buffer. JSON serialization via Jackson. Accept inbound order submissions.

Phase 5 — React Frontend

Depth chart, trade tape, manual trading interface. WebSocket client connecting to Netty backend.

Phase 6 — Benchmarking and Polish

JMH benchmarks for throughput and latency. End-to-end measurement. Resume bullet points. Documentation.

9. Resume Artifacts (Target)

Upon completion, this project should demonstrate:

- Low-latency systems design with the LMAX Disruptor pattern
- Lock-free, zero-allocation pipeline architecture
- Financial domain knowledge (limit order books, price-time priority, FIX protocol)
- Single-threaded matching engine design eliminating contention
- Mechanical sympathy (cache-line awareness, false sharing prevention)
- Full-stack delivery: Java backend with live React frontend
- Quantified performance characteristics via JMH benchmarks